



Indian Institute of Technology, Roorkee
Technical Report

Retrieval Integrity Auditor for RAG Systems

CONTACT INFO:

1. **Name** : Lakshya Gupta
2. **Contact Number** : 9837348350
3. **Branch**: Geological Technology
4. **Email ID**: lakshya_g@es.iitr.ac.in

1. Executive Summary

Retrieval-Augmented Generation (RAG) systems retrieve documents before a language model generates an answer. When that retrieval step fails — pulling irrelevant chunks, missing critical evidence, or over-relying on the single top match — the language model often hides the failure behind fluent, confident prose. Most evaluation today inspects only the **final answer**, never the **retrieval** that produced it. Enterprises are therefore exposed to compliance errors, operational mistakes, and decisions made on partial evidence, with no visibility into *why*.

The **Retrieval Integrity Auditor** is a tool that audits the retrieval step *before* generation. Given a user query and the chunks a retriever returned, it:

1. Decomposes the query into the distinct sub-aspects that must be answered.
2. Measures how well each aspect is covered by the retrieved chunks.
3. Classifies every retrieved chunk as **supporting**, **partial**, or **noise**.
4. Searches the full knowledge base for relevant evidence that *should* have been retrieved but was missed.
5. Produces a transparent **Retrieval Integrity Score (0–100)**.
6. Explains the failures in plain English and recommends concrete fixes.
7. Optionally evaluates retrieval against a labeled ground truth, returning **precision, recall, and F1**.

Every output is available both as **machine-readable JSON** (for automation and CI pipelines) and as a **human-readable report and interactive dashboard** (for non-technical reviewers). The system runs fully offline with no API keys, no GPU, and completes an audit in seconds.

2. Problem Statement & Motivation

2.1 The hidden failure mode

A RAG pipeline has two stages: **retrieve**, then **generate**. The generation stage is fluent by design, which means it can produce a plausible answer even when the retrieved context is incomplete or wrong. The retrieval stage fails silently in several characteristic ways:

- **Irrelevant retrieval** - chunks that are topically close but do not answer the question (the "keyword trap").
- **Missing evidence** - the document that actually answers the question exists in the knowledge base but ranks below the top-k cutoff.
- **Top-1 over-reliance** - one strong match masks the fact that other aspects of the query were never covered.
- **Unexplained ranking** - no insight into *why* the chosen chunks were ranked as they were, or what to change.

2.2 Why this matters to enterprises

In regulated and high-stakes settings legal, healthcare, finance, HR, compliance — an answer built on partial evidence is worse than no answer, because it carries the appearance of authority. The organisation needs to know, **before** an answer is shown to a user, whether the correct evidence was actually retrieved. That requires measuring evidence **coverage**, detecting **missing context**, isolating **retrieval noise**, and **explaining** ranking decisions — exactly the gaps this project addresses.

2.3 Scope

In line with the track guidance, the project deliberately does **not** train new embedding models, does not attempt perfect retrieval, and is not a full production RAG platform. It focuses on **auditing retrieval quality, measuring coverage, and explaining failures**.

3. System Overview

3.1 Inputs

Input	Description
Knowledge base	One or more PDF/TXT documents, uploaded and indexed
User query	The natural-language question

Input	Description
Retrieval output	Top-k retrieved chunks, each with chunk_id, text, rank, similarity_score, doc_id
Ground truth <i>(optional)</i>	Labeled relevant chunk_ids and/or doc_ids for quantitative evaluation

3.2 Outputs

Output	Description
Integrity Score (0–100)	Transparent, formula-based completeness score
Coverage analysis	Per-aspect status: COVERED / PARTIAL / MISSING
Evidence classification	Per-chunk label: SUPPORTING / PARTIAL / NOISE
Missing-evidence detection	KB chunks that should have been retrieved, with similarity and reason
Recommendations	Typed, actionable fixes (query rewrite, chunking, hybrid search, threshold, reranking)
Ground-truth metrics <i>(optional)</i>	Precision, recall, F1, plus hit / noise / missed-relevant breakdown
Reports	Machine-readable JSON and a self-contained HTML report

3.3 Architecture

The system is a **FastAPI** backend serving a dependency-free **vanilla JavaScript** frontend. Documents are embedded with the open-source all-MiniLM-L6-v2 sentence-transformer (384-dimensional vectors) and indexed in a **FAISS** IndexFlatL2 held in memory. An optional free LLM (Ollama, Groq, or Gemini) enriches the natural-language output; if none is configured, a built-in rule-based engine produces the same structured results with **zero external dependencies**.

4. The Audit Pipeline (Step by Step)

A single call to POST /audit runs an eight-step pipeline. Each step is an independent, testable module.

Step 1 - Aspect decomposition (aspect_decomposer.py).

The query is broken into 3–6 distinct sub-aspects, each a short phrase representing one information requirement. An LLM is used when available; otherwise a rule-based decomposer splits on conjunctions ("and", "also", "as well as") and question boundaries. Example: *"How many days of annual leave do employees get and what is the process to apply?"* → ["How many days of annual leave do employees get", "what is the process to apply for leave?"].

Step 2 - Coverage matrix (coverage_matrix.py).

Every aspect and every retrieved chunk is embedded, and a cosine-similarity score is computed for each (aspect × chunk) pair. The result is a dense matrix of values in [0, 1] — the foundation for both the coverage analysis and the visual heatmap.

Step 3 - Per-aspect coverage status.

For each aspect, the highest-scoring chunk determines its status:

Best similarity	Status
≥ 0.40	COVERED
0.24 – 0.40	PARTIAL
< 0.24	MISSING

Step 4 - Chunk classification (evidence_classifier.py).

Each retrieved chunk is labeled by its best similarity to *any* aspect:

Max similarity	Classification
≥ 0.40	SUPPORTING
0.25 – 0.40	PARTIAL
< 0.25	NOISE

Noise chunks carry an explicit reason string, making the judgement auditable.

Step 5 - Missing-evidence detection (`missing_detector.py`).

For every MISSING aspect, the system embeds the aspect phrase and searches the **full FAISS index** (not just the retrieved set). It returns the best candidate chunk that was *not* already retrieved, with a similarity proxy and a reason ("ranked below the top-k retrieval cutoff"). This is what lets the tool say *"the answer exists in `knowledge\_base.txt`, chunk c19 — your retriever just missed it."*

Step 6 - Integrity score (`scorer.py`).

A transparent, fully explainable formula (Section 5).

Step 7 - Explanation & recommendations (`explainer.py`).

A plain-English explanation of why retrieval was insufficient, plus 3–5 typed recommendations. Uses the LLM when available, with a rich rule-based fallback that covers query rewriting, chunking, hybrid (dense + BM25) search, similarity thresholds, and cross-encoder reranking.

Step 7b - Ground-truth evaluation (*optional*) (`ground_truth.py`).

If labeled relevant chunks are supplied, the system computes precision, recall, and F1, and reports which relevant chunks were missed (false negatives) and which retrieved chunks were noise (false positives).

Step 8 - Report assembly.

All results are packaged into a JSON response and a self-contained HTML report.

5. The Retrieval Integrity Score

The score is deliberately **formula-based and transparent** — never a black box — so a reviewer can reproduce and trust it.

Design rationale.

1. **Coverage is the dominant term.** Completeness of evidence is the core question, so the base score is driven entirely by the fraction of aspects fully covered.
2. **Noise is penalised proportionally.** Retrieving irrelevant chunks wastes the top-k budget and risks misleading generation; the penalty scales with the share of retrieved chunks that are noise (up to -20).

3. **Partial coverage is penalised lightly.** An aspect that is touched but not fully answered is better than a miss but worse than full coverage (−5 each).

Interpretation bands: 0–40 -> *Low integrity*; 41–70 -> *Medium integrity*; 71–100 -> *High integrity*. The dashboard renders the score on an animated gauge colour-coded to these bands.

Worked example. Two aspects, both covered (base = 100); three retrieved chunks, one of them noise (noise_ratio = 0.33 -> penalty = 6.6); no partial aspects. Final score ≈ **93/100** - high integrity, but flagged for one noise chunk.

6. Visualization

The mandatory visualization requirements are met by an interactive dashboard:

- **Aspect × Chunk coverage heatmap.** Rows are query aspects, columns are retrieved chunks, and each cell is colour-graded by similarity. The 0.40 coverage threshold is marked on the legend.
- **Missing-aspect highlighting.** Aspects with MISSING status are flagged directly on the heatmap row, drawing the eye to the gaps.
- **Clickable drill-down.** Clicking any heatmap cell opens a detail drawer showing the full chunk text, exact similarity, classification, the aspects it covers, and any noise reason satisfying the drill-down-to-evidence requirement.
- **Supporting tabs.** *Evidence* (filter/search chunks by classification), *Missing* (candidate chunks the retriever should have surfaced), and *Fixes* (the recommendations) round out the view.
- **Score panel.** Animated gauge, component metrics (aspects, covered, missing, supporting, noise), coverage and noise bars, and — when ground truth is supplied — a precision / recall / F1 strip.

The dashboard also includes audit history (stored locally), JSON export, and a shareable link, supporting real enterprise review workflows.

7. Conclusion

The Retrieval Integrity Auditor makes the invisible failure mode of RAG systems **visible, measurable, and explainable**. Instead of reporting "retrieval failed," it tells a reviewer exactly which aspects of the query were missed, which retrieved chunks were noise, which documents should have been retrieved, how complete the retrieval was on a transparent 0–100 scale, and when ground truth is available — the precise precision, recall, and F1. It does so with an interactive, drill-down dashboard and machine-readable output, running fully offline with no external dependencies. The result is exactly what the track calls for: visibility into whether the correct evidence was retrieved **before** generation occurs.